Backend-Workflow:

Data model : ( )

users

- id (uuid, PK)
- email (unique)
- password_hash (nullable for social login)
- name
- role (enum: buyer, vendor_admin, admin)
- created_at, updated_at
- last_login

user_profiles

- id, user_id (FK)
- phone
- shipping_addresses (json) — or separate addresses table
- billing_methods (json) — or token refs to payment gateway
- preferences

vendors

- id (uuid)
- owner_user_id (FK to users)
- company_name
- status (pending / approved / suspended)
- api_credentials (encrypted json)
- created_at, updated_at

products

- id (uuid)
- vendor_id (FK)
- sku (unique)

- title, description, attributes (jsonb)

- price_cents, currency
- images (jsonb: s3 urls)
- category_id
- stock (int) — canonical stock (source of truth)
- is_active (bool)
- created_at, updated_at
- search_indexed (bool)

categories

- id, name, slug, parent_id

product_inventory_logs

- id, product_id, change, reason, meta (jsonb), created_at (for audit)

carts

- id, user_id, items (jsonb) — or cart_items table
- updated_at

cart_items (if normalized)

- id, cart_id, product_id, sku, qty, price_at_add_cents

orders

- id (uuid), user_id, vendor_id (nullable if marketplace order with multiple vendors use order_items grouping)
- total_amount_cents, currency
- status (pending, paid, processing, shipped, delivered, cancelled, refunded)
- shipping_address (jsonb)
- payment_method (jsonb)
- created_at, updated_at

order_items

- id, order_id, product_id, sku, qty, price_cents, vendor_id
- fulfillment_status, tracking (jsonb)

payment_transactions

- id, order_id, gateway (stripe…), gateway_payment_id, amount_cents, status, raw_response (jsonb), created_at

refunds

- id, order_id, payment_transaction_id, amount_cents, status, created_at

payouts

- id, vendor_id, period_start, period_end, amount_cents, status, createdat, metadata

webhooks

- id, type, payload (jsonb), status, attempts, last_attempted_at

notifications

- id, user_id, channels (email/sms/push), payload (jsonb), status, sent_at

vendor_product_sync_logs

- id, vendor_id, product_count, status, errors (jsonb), timestamp

admin_audit_logs

- id, admin_id, action, target, details (jsonb), timestamp

Core workflows (detailed)

1) Buyer signup / auth

1. Buyer registers (email/password) or OAuth (Google).
2. Create `users` + `user_profiles`.
3. Send email verification (link with token).
4. Issue JWT + refresh token stored in DB/Redis with expiry.

Security: store passwords with bcrypt/argon2; rate-limit login; lockout after N fails.

2) Product ingestion (Vendor)

- Vendor obtains API key (issued by system; stored encrypted).
- Vendor posts product catalog endpoint: POST /vendor/products/bulk
  - Authenticate via API key (HMAC or JWT).
  - Accept JSON or CSV; images via S3 upload URLs.
- Worker validates payload, upserts `products`, and writes `vendor_product_sync_logs`.
- Admin can approve/curate via Admin UI; `is_active` controls marketplace visibility.

3) Browsing & search

- Buyers call GET /products?category=&q=&filters...
- API reads from DB + uses Redis cache; for advanced queries use search index (ES) — sync products to ES via worker when product changes.

4) Cart → Checkout → Payment (critical transactional flow)

1. Buyer adds to cart. On "checkout" server:
   - Validate prices and availability by fetching current `products.stock` and price.
   - **Reserve stock** using a reservation table or Redis lock:
     - Create `inventory_reservations` with TTL (e.g., 15 minutes).
     - Decrement **available** stock logically or mark reserved qty.
   - Create `orders` with status `pending`.
2. Start payment with gateway. Save `payment_transactions` with status `initiated`.
3. When payment succeeds (webhook/callback):
   - Mark payment_transaction `paid`.
   - Move order to `processing` and finalize reservation → deduct actual `product.stock` within a DB TX to avoid race.
   - Send order event to fulfillment queue.
   - Emit notifications.
4. If payment fails/timeout: release reservation, set order `cancelled`.

Important: use DB transactions + SELECT ... FOR UPDATE on product rows when confirming stock to avoid overcommit. Use idempotency keys for payment endpoints.

5) Order fulfillment & tracking

- Orders for each vendor go to fulfillment queue (webhook or API call to vendor).
- Vendors confirm & push tracking info → update order_item.tracking and order status.
- Periodic worker reconciles missing tracking updates and escalates.

6) Returns & refunds

- Buyer initiates return → create return request record, move item to `return_pending`.
- Admin or vendor approves → create refund via payment gateway, track refund transaction.
- Update order status & notify buyer.

7) Payouts to vendors

- Worker runs scheduled job (e.g., daily/weekly) to aggregate settled orders per vendor (completed & not yet paid).
- Create `payouts` record and transfer via payout rails (bank API or Stripe Connect).
- Update `payouts.status` and vendor ledger.

8) Notifications

- All events create a notification job (e.g., order_placed, shipped)
- Notification worker decides channels (email/SMS/push), composes templates, sends via provider, retries on failure.

APIs — REST endpoints (core, concise)

Authentication

- POST /auth/register {email, password}
- POST /auth/login {email, password} → returns access_token, refresh_token
- POST /auth/oauth/callback
- POST /auth/refresh {refresh_token}
- POST /auth/logout

Buyer / Catalog

- GET /products
- GET /products/:id
- GET /categories
- POST /wishlist (auth)
- GET /wishlist
- POST /cart, PUT /cart/:item_id, DELETE /cart/:item_id
- POST /checkout {cart_id, shipping_address_id, payment_method_id}
- GET /orders (user)
- GET /orders/:id

Vendor

- POST /vendor/register
- POST /vendor/api-keys (create/regenerate)
- POST /vendor/products (single)
- POST /vendor/products/bulk
- PUT /vendor/products/:id
- GET /vendor/orders
- POST /vendor/fulfillment/update (webhook endpoint)

Admin

- GET /admin/vendors (approve/reject)
- POST /admin/products/:id/suppress
- GET /admin/orders (search/filter)
- POST /admin/orders/:id/refund

Payments & Webhooks

- POST /webhooks/payments (idempotent)
- POST /webhooks/vendor-fulfillment
- POST /webhooks/search-sync (optional)

Tech Stack Overview

- **Backend:** Node.js (Express.js or Fastify — we'll use Express for this plan)
- **Database:** PostgreSQL (via Prisma ORM or Sequelize — Prisma recommended)
- **Cache/Session Store:** Redis
- **Authentication:** JWT + OAuth (for social login)
- **File Storage:** AWS S3 / Cloudinary (for product images)
- **Payments:** Stripe / Razorpay example
- **Notifications:** Redis-based Pub/Sub + Email service (e.g., Nodemailer)
- **Search (Phase 2):** PostgreSQL full-text search or Elasticsearch

---

Backend Workflow Overview

Buyers

1. Account & Profile

- **Signup/Login (Email/OAuth)** → `/api/auth/register`, `/api/auth/login`
- **Manage profile** → `/api/user/profile`
- **Address CRUD** → `/api/user/address`
- **Payment methods (saved)** → `/api/user/payment-methods`

2. Product Discovery

- **Fetch categories** → `/api/categories`
- **Browse products** → `/api/products`
- **Search/filter** → `/api/products/search`
- **Wishlist CRUD** → `/api/wishlist`

3. Product Detail & Cart

- **Get product details** → `/api/products/:id`
- **Add/update/remove from cart** → `/api/cart`

4. Checkout & Payment

- **Checkout endpoint** → `/api/checkout`
- **Stripe/Razorpay payment webhook** → `/api/payments/webhook`
- **Order confirmation** → `/api/orders/confirm`

5. Post-Purchase

- **Track orders** → `/api/orders/:id`
- **View past orders** → `/api/orders/history`
- **Return/refund request** → `/api/orders/refund`
- **Notifications (Redis Pub/Sub)** → real-time status updates

---

Vendor

1. Vendor Onboarding

- **Register vendor account** → `/api/vendor/register`
- **API key generation** → `/api/vendor/api-key`
- **Upload product catalog (via API)** → `/api/vendor/products/upload`

2. Product Management

- **CRUD products** → `/api/vendor/products`

- **Activate/deactivate** → `/api/vendor/products/:id/status`
- **Sync inventory via API** → `/api/vendor/inventory/sync`

3. Order Management

- **Fetch new orders** → `/api/vendor/orders`
- **Confirm/update status** → `/api/vendor/orders/:id`
- **Push fulfillment details** → `/api/vendor/orders/fulfill`

4. Payouts

- **View completed orders** → `/api/vendor/payouts`
- **Track payout status** → `/api/vendor/payouts/:id`

👨‍💻 Admin

1. Vendor & Catalog Management

- **Approve/reject vendors** → `/api/admin/vendors`
- **Curate products** → `/api/admin/products`

2. Order Oversight

- **View all orders** → `/api/admin/orders`
- **Cancel/refund** → `/api/admin/orders/:id`

3. Fulfillment Oversight

- **Monitor fulfillment** → `/api/admin/fulfillment`

4. System Monitoring

- **API sync health** → `/api/admin/system/health`
- **Inventory mismatches** → `/api/admin/system/inventory`

⚙️ System

1. Payments

- **Handle payments via Stripe/Razorpay SDK**
- **Trigger refunds**
- **Record payouts to vendors**

2. Shipping & Fulfillment

- **Integrate with courier APIs**
- **Sync tracking details → Redis → notify buyer**

3. Notifications

- **Redis Pub/Sub** for real-time updates
- **Email/SMS via Nodemailer/Twilio**

4. Search & Recommendations (Phase 2)

- **Redis caching** for trending products
- **PostgreSQL search or Elasticsearch integration**

📁 Folder Structure

```
backend/
│
├── src/
│   ├── config/
│   │   ├── db.js              # PostgreSQL connection (via Prisma/Sequelize)
│   │   ├── redis.js           # Redis connection setup
│   │   ├── env.js             # Environment variables loader
│   │
│   ├── models/                # Prisma models or Sequelize schemas
│   │   ├── user.model.js
│   │   ├── vendor.model.js
│   │   ├── product.model.js
│   │   ├── order.model.js
│   │   ├── category.model.js
│   │   ├── payment.model.js
│   │   ├── wishlist.model.js
│   │
│   ├── controllers/           # Logic for each route
│   │   ├── auth.controller.js
│   │   ├── user.controller.js
│   │   ├── product.controller.js
│   │   ├── order.controller.js
```

```
│   │   ├── vendor.controller.js
│   │   ├── admin.controller.js
│   │
│   ├── routes/              # Define all API routes
│   │   ├── auth.routes.js
│   │   ├── user.routes.js
│   │   ├── product.routes.js
│   │   ├── order.routes.js
│   │   ├── vendor.routes.js
│   │   ├── admin.routes.js
│   │
│   ├── middlewares/
│   │   ├── auth.middleware.js   # JWT verification
│   │   ├── role.middleware.js   # Buyer/Vendor/Admin role guard
│   │   ├── error.middleware.js  # Error handling
│   │
│   ├── services/
│   │   ├── auth.service.js
│   │   ├── payment.service.js
│   │   ├── redis.service.js     # Redis helpers for caching/session/pub-sub
│   │   ├── notification.service.js
│   │   ├── vendor.service.js
│   │   ├── order.service.js
```

```
│   │
│   ├── utils/
│   │   ├── response.util.js     # success/error response formatters
│   │   ├── token.util.js        # JWT helpers
│   │   ├── validator.util.js    # Joi/Zod schema validators
│   │   ├── logger.util.js       # Winston/Pino logger
│   │
│   ├── events/                  # Redis Pub/Sub channels
│   │   ├── order.events.js      # handles order-related notifications
│   │   ├── notification.events.js
│   │
│   ├── app.js                   # Express app configuration
│   └── server.js                # Entry point
│
├── prisma/                      # if using Prisma ORM
│   └── schema.prisma
│
├── .env
├── package.json
└── README.md
```

🔁 Redis Usage

Purpose	Redis Data Type	Example Key
Caching product list	String / JSON	`products:category:electronics`
Session storage	Hash	`session:user:123`
Real-time notifications	Pub/Sub	`notifications:buyer`
Rate limiting	Counter	`rate:login:ip`
Order status updates	Pub/Sub	`orders:status`

🧩 Request Flow Example (Buyer placing order)

1. `POST /api/cart/checkout`
2. Validate JWT → Fetch cart → Calculate total
3. Create payment intent (Stripe/Razorpay)
4. On success → Create order in PostgreSQL
5. Publish event → Redis: `orders:created`

6. Vendor service (subscriber) listens → Fetches order → Accepts

7. Redis publishes `orders:accepted` → Buyer notified via socket/email